2

Laboratory 2

2. Detection of foreground moving objects

What are foreground objects?

These are the objects that are important to us (in the context of the application under consideration). Most often, these are: people, animals, cars (or other vehicles), and luggage (potential bombs). The definition is therefore closely linked to the target application.

Is foreground object segmentation a moving object segmentation?

No. First, an object that has stopped may still be of interest to us (e.g. a person standing in front of a pedestrian crossing). Second, there are many moving elements in the scene that are not relevant to us (e.g. flowing water, a fountain, moving trees or bushes). It is also worth noting that motion is often analysed in image processing. Therefore, the detection of foreground objects can (and should) be supported by the detection of moving objects.

The simplest method of detecting moving objects is based on subtracting consecutive (adjacent) frames. In this exercise, we will implement simple subtraction of two frames, combined with binarisation, connected component labelling, and analysis of the obtained objects. Finally, we will try to treat the result of the subtraction as the outcome of foreground object segmentation and examine the quality of this segmentation.

2.1 Image sequence loading

The sequences used come from a dataset available on the changedetection.net website. In case of problems with accessing the service, the dataset is shared on the UPeL platform. The dataset contains sequences with labels, i.e. each frame of the image has an assigned reference mask of objects (ground truth), in which each pixel belongs to one of five categories: background (0), shadow (50), outside the region of interest (85), unknown motion

(170), and foreground objects (255) – the corresponding greyscale levels are given in parentheses.

For the purposes of this exercise, we are only interested in separating foreground objects from the remaining categories. Additionally, the folder contains a region of interest (ROI) mask and a text file with the time interval for which the results should be analysed (`temporalROI.txt`) – details are provided later in the exercise.

Exercise 2.1 Load the sequences. ■

2.2 Frame subtraction and Binarisation

For the purpose of detecting moving elements, we subtract the previous frame from the current frame; note that in practice we compute the absolute value of this difference. Then, in order to perform binarisation, we need to determine a threshold and choose it so that the objects are relatively clear. Please pay attention to artifacts related to compression.

R To avoid problems with unsigned number subtraction (*uint8*), perform a conversion to type *int* – `IG = IG.astype('int')`.

```
(T, thresh) = cv2.threshold(D,10,255,cv2.THRESH_BINARY)
# D -- input array
# 10 -- threshold value
# 255 -- maximum value to use with the THRESH_BINARY and THRESH_BINARY_INV
    thresholding types
# cv2.THRESH_BINARY -- thresholding type
# T -- our threshold value
# thresh -- output image
```

R The first argument of the returned values by the `cv2.threshold` function is the binarisation threshold (this is useful when using automatic threshold determination, e.g. with the Otsu or triangle method). See the OpenCV documentation for details.

Exercise 2.2 Perform the appropriate operations to obtain a binarised image. ■

2.3 Morphological operations

Morphological operations are simple binary image processing operations (sometimes also used for greyscale images) that modify the shape of objects based on the so-called structuring element (e.g. 3×3).

- erosion (*erosion*) – shrinks objects: makes them smaller, removes small bright noise, can break thin connections,
- dilation (*dilation*) – expands objects: makes them larger, closes small gaps/holes, merges nearby fragments,
- opening (*opening* = erosion → dilation) – removes small objects/noise, smooths the contours,
- closing (*closing* = dilation → erosion) – fills small holes and gaps, connects cracks.

The resulting image is quite noisy. The goal is to obtain as clearly visible a silhouette as possible with minimal disturbances. To improve the result, it is worth adding a median filtering step (before the morphological operations) and, if necessary, adjusting the binarisation threshold.

Exercise 2.3 Perform filtering using erosion and dilation (`erode` and `dilate` from OpenCV).
■

2.4 Labelling and simple analysis

In the next step, the obtained result must be filtered. For this purpose, indexing (assigning labels to groups of connected pixels) will be used, along with the calculation of parameters for these groups. The function connectedComponentsWithStats is used for this. Function call:

```
retval, labels, stats, centroids =  cv2.connectedComponentsWithStats(B)
# retval --  total number of unique labels
# labels -- destination labelled image
# stats -- statistics output for each label, including the background label.
# centroids -- centroid output for each label, including the background label.
```

R When displaying the *labels* image, you need to set the format properly and add scaling.
You can use information about the number of objects found for scaling.

```
cv2.imshow("Labels", np.uint8(labels / retval * 255))
```

Then display the bounding box, area, and index for the largest object. Below is a sample solution to the task. Please run it and, if possible, try to optimise it.

```
I_VIS = I # copy of the input image

if (stats.shape[0] > 1):   # are there any objects

  tab = stats[1:,4] # 4 columns without first element
  pi = np.argmax( tab ) # finding the index of the largest item
  pi = pi + 1 # increment because we want the index in stats, not in tab
  # drawing a bbox
  cv2.rectangle(I_VIS,(stats[pi,0],stats[pi,1]),(stats[pi,0]+stats[pi,2],stats[pi
      ,1]+stats[pi,3]),(255,0,0),2)
  # print information about the field and the number of the largest element
  cv2.putText(I_VIS,"%f" % stats[pi,4],(stats[pi,0],stats[pi,1]),cv2.
      FONT_HERSHEY_SIMPLEX,0.5,(255,0,0))
  cv2.putText(I_VIS,"%d" %pi,(np.int(centroids[pi,0]),np.int(centroids[pi,1])),cv2.
      FONT_HERSHEY_SIMPLEX,1,(255,0,0))
```

Comments on the sample solution:

- `stats.shape[0]` is the number of objects. Since the function also counts the object with index 0 (i.e. background), in the conditional function check if there are objects where the condition is > 1.
- the next two lines are the calculation of the maximum index from column number 4 (field).
- the resulting index should be incremented, as we have omitted element 0 (background) from the analysis.
- We use the `rectangle` function from OpenCV to draw a surrounding rectangle in the image. The syntax `"%f" % stats[pi,4]` allows us to output the value in the appropriate format (f – float). The next parameters are the coordinates of the two opposite vertices of the rectangle. Then the colour in format (B,G,R) and finally the line thickness. See the function documentation for details.

- The function `putText` is used to output text on the image. The coordinates of the lower left vertex are given to determine the position of the text box. The next parameters are the font (full list in the documentation), size and colour.

Exercise 2.4 Use the function `cv2.connectedComponentsWithStats` to label and calculate the parameters of the resulting objects. Display the surrounding rectangle, field and index for the largest object. Based on the tips and examples in the text above, try to optimise the solution. ■

2.5 Evaluation of foreground object detection results

To evaluate a foreground object detection algorithm, the object mask it returns must be compared with a reference mask (the ground truth). The comparison is carried out at the level of individual pixels. If shadows are excluded (as assumed at the outset), four situations are possible:

- TP – *true positive* – a pixel belonging to a foreground object is detected as a pixel belonging to a foreground object,
- TN – *true negative* – a pixel belonging to the background is detected as a pixel belonging to the background,
- FP – *false positive* – a pixel belonging to the background is detected as a pixel belonging to a foreground object,
- FN – *false negative* – a pixel belonging to an object is detected as a pixel belonging to the background.

Based on these values, a number of measures can be calculated. In the following, we will use three: precision (P), recall (R), and the so-called $F1$ measure. These are defined as follows:

$$P = \frac{TP}{TP+FP} \tag{2.1}$$

$$R = \frac{TP}{TP+FN} \tag{2.2}$$

$$F1 = \frac{2PR}{P+R} \tag{2.3}$$

The F1 score is in the range $[0;1]$, with the higher its value, the better result.

Exercise 2.5 Implement the computation of the metrics P, R and $F1$.

R However, we only perform the calculation if a valid reference map is available. To do this, check the dependence of the frame counter and the value from the `temporalROI.txt` file – it must be within the range described there.

■

Exercise 2.6 To summarise, you need to perform moving object detection:

1. Load the `pedestrian` sequence, and then `highway` and `office`.
2. Perform change detection: frame differencing and binarisation.
3. Remove noise (e.g. using a median or Gaussian filter).

4. Apply morphological operations.
5. Perform connected component labelling.
6. Carry out the evaluation.

Exercise completion

In order to have the exercise accepted, after completing all the tasks in this laboratory, report your readiness to the teacher. Once it has been approved by the teacher, place the script with the `.py` or `.ipynb` extension in the appropriate location in the course resources on UPeL. ■

R Information on the following classes.

1. In further exercises we will learn more algorithms and functionalities of the Python language. However, we do not put special emphasis on learning the Python language, but on using in practice the successive algorithms appearing in the lectures. The subject Advanced Vision Systems is not a Python course!
2. The solutions presented should always be considered as examples. The problem can certainly be solved differently, and sometimes better.

2.6 Example results of the algorithm for foreground moving object detection

In order to better verify the different steps of the proposed method for foreground moving object detection, an example result for an image with index 350 is presented.

Figure 2.1: An example of the result of each stage of the algorithm. From left: input image with bounding box, image after binarisation, image after median filtering and morphological operations (erode, dilate), image representing labels after labelling, reference image – ground truth.